

Chapter 10 — Common Mistakes, Best Practices Checklist, and Glossary

This chapter is your safety net. Read it before submitting a PR. Use the checklist to verify your implementation. Use the glossary when you encounter an unfamiliar term.

10.1 The 15 Most Common Mistakes

Each mistake includes: what the wrong code looks like, what the correct code looks like, and why it matters.

Mistake 1: Fetching data in the page and prop-drilling it down

```
// ✗ WRONG — page fetches and prop-drills
function CrudPage() {
  const { data: activities } = useGetCategories();
  return <ItemForm activities={activities} />;
}
function ItemForm({ activities }) { ... }

// ☑ CORRECT — ItemForm fetches its own data
function ItemForm() {
  const { data: categoriesResponse } = useGetCategories();
  const categories = categoriesResponse?.ResponseActivities ?? [];
}
```

Why it matters: React Query caches by query key. Calling `useGetCategories()` in `ItemForm` returns cached data instantly — no extra network request. Prop-drilling adds complexity with zero benefit. If you later add another component that needs categories, you'd have to thread the prop through every intermediate component.

Best practice: Components own their own data fetching. The page hook does NOT pass query results as props to components that can fetch their own data.

Mistake 2: Putting business logic in the page component

```
// ✗ WRONG — logic in the page, untestable without rendering
function CrudPage() {
  const [modalOpen, setModalOpen] = useState(false);
  const [editCtx, setEditCtx] = useState(null);
  const handleEdit = (id) => {
    const dto = items.find(i => i.StudentNumber === id);
    setEditCtx(buildEditContext(dto));
    setModalOpen(true);
  };
}
```

```

    };
    return <ItemTable onEdit={handleEdit} />;
  }

  // ☒ CORRECT – logic in the hook, testable with renderHook
  function CrudPage() {
    const { modalOpen, editCtx, handleEdit } = useCrudPage();
    return <ItemTable onEdit={handleEdit} />;
  }

```

Why it matters: Logic in the page cannot be unit tested without rendering the component and simulating user interactions. Logic in a custom hook can be tested with `renderHook` — no UI rendering needed.

Mistake 3: Writing TypeScript types manually instead of using `z.infer`

```

// ☒ WRONG – two sources of truth that can drift apart
const ItemDtoSchema = z.object({
  StudentName: z.string(),
  StudentNumber: z.number(),
});
type ItemDto = {
  StudentName: string;
  StudentNumber: number;
  // If you add a field to the schema but forget the type, TypeScript won't catch it
};

// ☒ CORRECT – single source of truth
const ItemDtoSchema = z.object({
  StudentName: z.string(),
  StudentNumber: z.number(),
});
type ItemDto = z.infer<typeof ItemDtoSchema>; // always in sync with the schema

```

Why it matters: When you add a field to the Zod schema, `z.infer` automatically adds it to the TypeScript type. With a manually written type, you have to remember to update it — and you will forget.

Mistake 4: Using `Promise.all` for multi-delete

```

// ☒ WRONG – one failure cancels all remaining deletions
await Promise.all(selectedIds.map(id => deleteMutation.mutateAsync({
  studentActivityId: id })));
// If item 3 of 5 fails, items 4 and 5 are never deleted

// ☒ CORRECT – all run, failures are counted and reported
const results = await Promise.allSettled(
  selectedIds.map(id => deleteMutation.mutateAsync({ studentActivityId: id })))

```

```
);
const failed = results.filter(r => r.status === "rejected").length;
if (failed > 0) toast.error(`${failed} of ${selectedIds.length} deletions
failed.`);
```

Why it matters: `Promise.all` fails fast — one rejection cancels everything. `Promise.allSettled` waits for all and reports results. Partial success is always better than no success for multi-item operations.

Mistake 5: Not using `useCallback` on handlers passed as props

```
// ✗ WRONG – new function reference every render → child re-renders
unnecessarily
function useCrudPage() {
  const handleEdit = (id: number) => {
    const dto = items.find(i => i.StudentNumber === id);
    dispatch({ type: "OPEN_EDIT", ctx: buildEditContext(dto) });
  };
  return { handleEdit };
}

// ☑ CORRECT – stable reference, child only re-renders when deps change
function useCrudPage() {
  const handleEdit = useCallback((id: number) => {
    const dto = items.find(i => i.StudentNumber === id);
    if (!dto) return;
    dispatch({ type: "OPEN_EDIT", ctx: buildEditContext(dto) });
  }, [items]); // items is a dependency because it's used inside
  return { handleEdit };
}
```

Why it matters: Every time `useCrudPage` re-renders (e.g., because `isFetching` changed), a new `handleEdit` function is created. Since it's passed as a prop to `<ItemTable>`, `ItemTable` sees a new prop reference and re-renders — even though nothing relevant changed. `useCallback` prevents this.

Mistake 6: Not using `useMemo` for the columns array

```
// ✗ WRONG – new columns array every render → TanStack Table re-initializes
// → sort order, filters, and pagination reset on every render
const columns = buildItemColumns({ actions });

// ☑ CORRECT – stable reference, TanStack Table only re-initializes when actions
changes
const columns = useMemo(
  () => buildItemColumns({ actions }),
  [actions]
);
```

Why it matters: TanStack Table detects a new `columns` reference as a configuration change and re-initializes — resetting sort order, column filters, and pagination to defaults. This causes jarring UX: the user sorts by name, the table refetches in the background, and the sort resets.

Mistake 7: Missing `enabled: !!params` on search-triggered queries

```
// ✗ WRONG – fires immediately on page load with no filters
// Loads potentially thousands of records before the user has set any filters
export const useGetItems = (params?: GetItemsParams) =>
  useQuery({
    queryKey: [...QUERY_KEYS.items, params],
    queryFn: () => getItems(params),
    // missing enabled!
  });

// ✓ CORRECT – only fires after the user fills in filters and clicks Search
export const useGetItems = (params?: GetItemsParams) =>
  useQuery({
    queryKey: [...QUERY_KEYS.items, params],
    queryFn: () => getItems(params),
    enabled: !!params, // undefined → false (disabled), any object → true
  })
```

Why it matters: Without `enabled: !!params`, the query fires on page load with `params: undefined`. The server receives a request with no filters and returns all records — potentially thousands. This is slow and unnecessary.

Mistake 8: Not awaiting `invalidateQueries`

```
// ✗ WRONG – modal closes before re-fetch completes, table shows stale data
onSuccess: () => {
  toast.success("Created.");
  queryClient.invalidateQueries({ queryKey: QUERY_KEYS.items }); // not awaited!
  // onSuccess returns immediately, modal closes, re-fetch hasn't started yet
},

// ✓ CORRECT – re-fetch is triggered before onSuccess returns
onSuccess: async () => {
  toast.success("Created.");
  await queryClient.invalidateQueries({ queryKey: QUERY_KEYS.items });
  // now the re-fetch has been triggered and the cache is being updated
},
```

Why it matters: Without `await`, the modal closes and the table briefly shows stale data. The user sees their newly created item missing from the table for a moment. With `await`, the re-fetch is triggered synchronously before the modal closes.

Mistake 9: Forgetting `.nullable()` for fields that can be null

```
// ✗ WRONG – Zod throws when server sends null for this field
const ItemDtoSchema = z.object({
  LastActivityDate: z.string(), // throws: "Expected string, received null"
});

// ☑ CORRECT – accepts null
const ItemDtoSchema = z.object({
  LastActivityDate: z.string().nullable(), // accepts "2024-03-15" or null
});
```

How to know which fields can be null: Check the C# model. If the property is `string?` (nullable reference type) or `DateTime?` (nullable value type), add `.nullable()` to the Zod schema.

Mistake 10: Using `as` instead of `satisfies` for column definitions

```
// ✗ WRONG – "as" suppresses TypeScript errors, hides bugs
{
  id: "actions",
  typo_field: "oops", // TypeScript doesn't catch this
} as ColumnDef<ItemRow>

// ☑ CORRECT – "satisfies" checks the type AND catches errors
{
  id: "actions",
  typo_field: "oops", // TypeScript ERROR: "typo_field" is not in
  ColumnDef<ItemRow>
} satisfies ColumnDef<ItemRow>
```

Why it matters: `as` is a type assertion — it tells TypeScript "trust me, this is the right type" and suppresses all errors. `satisfies` is a type check — it verifies the object matches the type and reports errors. Always prefer `satisfies` over `as` when you want TypeScript to verify your code.

Mistake 11: Overriding URL params in the page hook instead of `select`

```
// ✗ WRONG – only the page hook gets the override
// Other components calling useGetSettings() get the unoverridden value
function useCrudPage() {
  const [searchParams] = useSearchParams();
```

```

    const { data: settings } = useGetSettings();
    const showWidget = parseBoolParam(searchParams.get("showWidget")) ??
settings?.showWidget;
    // showWidget is only available here – must be prop-drilled to every component
that needs it
  }

// ☑ CORRECT – all consumers of useGetSettings() automatically get the override
export const useGetSettings = () => {
  const [searchParams] = useSearchParams();
  return useQuery({
    queryKey: QUERY_KEYS.settings,
    queryFn: getSettings,
    select: (data) => ({
      showWidget: parseBoolParam(searchParams.get("showWidget")) ??
data.showWidget,
    }),
  });
};

```

Why it matters: The `select` option runs for every consumer of the hook. Every component calling `useGetSettings()` automatically gets the URL-overridden value. No prop drilling, no extra state.

Mistake 12: Using separate `useState` for coupled state

```

// ✗ WRONG – two separate state updates can cause intermediate renders
const [modalOpen, setModalOpen] = useState(false);
const [editCtx, setEditCtx] = useState<ItemEditContext | null>(null);

const handleEdit = (id) => {
  setModalOpen(true); // render 1: modalOpen=true, editCtx=null → modal renders
with no context!
  setEditCtx(ctx);    // render 2: modalOpen=true, editCtx=ctx → correct
};

// ☑ CORRECT – atomic update, single render
const [modal, dispatch] = useReducer(modalReducer, initialModalState);

const handleEdit = (id) => {
  dispatch({ type: "OPEN_EDIT", ctx }); // single render with both changes
};

```

Why it matters: React 18 batches state updates in event handlers, but in async callbacks or third-party event systems, two separate `setState` calls can cause an intermediate render. `useReducer` guarantees atomic updates — the state changes in a single dispatch.

Mistake 13: Forgetting `required_error` for Zod form fields

```
// ✗ WRONG – no error shown when field is never touched (value is undefined)
activityId: z.string().min(1, "Activity is required"),
// .min(1) only fires when the value is "" (empty string)
// When the field is never touched, value is undefined – .min(1) doesn't fire

// ☑ CORRECT – error shown even when field is never touched
activityId: z
  .string({ required_error: "Activity is required" }) // fires when value is
  undefined
  .min(1, "Activity is required"), // fires when value is ""
```

Why it matters: Without `required_error`, a user who never touches the Activity field and clicks Submit sees no error for that field — they don't know what's wrong.

Mistake 14: Not resetting the form when the modal re-opens

```
// ✗ WRONG – form keeps previous values when modal opens for a different item
useEffect(() => {
  reset(defaultValues);
}, [editCtx]); // missing `open` in deps – doesn't fire when modal re-opens for
same item

// ☑ CORRECT – reset whenever the modal opens OR the context changes
useEffect(() => {
  if (!open) return; // don't reset when closing
  reset(defaultValues);
}, [open, reset, defaultValues]);
```

Why it matters: Without the `open` dependency, editing Student A, closing, then editing Student A again (same `editCtx`) doesn't trigger the reset — the form might have dirty values from the previous edit.

Mistake 15: Skipping `zodParse` on API responses

```
// ✗ WRONG – no runtime validation, silent failures
const getItems = async (params) => {
  const response = await apiClient.get("/api/...", { params });
  return response.data as GetItemsResponse; // "as" is a lie – no validation
};
// If the server sends null for a required field, it silently enters your state
// and crashes 5 function calls later with a confusing error

// ☑ CORRECT – validated at the boundary
const getItems = async (params) => {
  const response = await apiClient.get("/api/...", { params });
  return zodParse(GetItemsResponseSchema, response.data, "getItems");
};
```

```
// If the server sends null for a required field, you get immediately:  
// "[getItems] Zod parse failed: Expected string, received null at path:  
StudentName"
```

Why it matters: TypeScript types are erased at runtime. `as GetItemsResponse` is a compile-time lie — it tells TypeScript to trust the data without actually validating it. Zod validates the actual runtime data at the boundary.

10.2 Best Practices Checklist

Use this checklist before submitting a PR for any new feature:

Architecture

- ☐ Each file has exactly one responsibility
- ☐ No business logic in the page component — it only calls the hook and renders
- ☐ No data fetching in the page hook that should be in a component
- ☐ Components fetch their own data (no prop-drilling query results)
- ☐ URL param overrides are in the query layer's `select` option

TypeScript

- ☐ All DTO types use `z.infer<typeof Schema>` — no manually written types
- ☐ All form value types use `z.infer<typeof FormSchema>`
- ☐ `import type` used for type-only imports
- ☐ `satisfies` used instead of `as` for type assertions where possible

Zod

- ☐ All API response schemas use `zodParse` in the service function
- ☐ Nullable fields have `.nullable()` in the schema
- ☐ Form schemas have `required_error` for required fields
- ☐ Form schemas have `.default()` for optional fields

React Query

- ☐ `enabled: !!params` on search-triggered queries
- ☐ `staleTime` set on reference data (categories, years, etc.)
- ☐ `await queryClient.invalidateQueries(...)` in mutation `onSuccess`
- ☐ Query keys centralized in `QUERY_KEYS` object with `as const`
- ☐ `toast.success` and `toast.error` in mutation callbacks

Performance

- ☐ `useMemo` on computed arrays/objects passed to children
- ☐ `useCallback` on all handlers passed as props
- ☐ `useMemo` on the `columns` array in the table component
- ☐ `useMemo` on the `actions` config in the table component

State Management

- ☐ Coupled state uses `useReducer` (not multiple `useState`)
- ☐ Independent state uses `useState`
- ☐ Reducer returns new objects (never mutates existing state)
- ☐ `default: return state` in every reducer

Forms

- ☐ `FormProvider` wraps the form in the modal
- ☐ `useFormContext` used in form fields (not prop drilling)
- ☐ `Controller` used for all Shadcn inputs (Select, Checkbox, Calendar, etc.)
- ☐ `useEffect` resets form on modal open
- ☐ Submit button disabled while `isPending`

Accessibility (WCAG)

- ☐ All interactive elements have `aria-label` or visible label
- ☐ Form fields have `aria-invalid` and `aria-describedby` for errors
- ☐ Error message elements have matching `id` for `aria-describedby`
- ☐ Status/empty state components use `role="status"` and `aria-live="polite"`
- ☐ Modal returns focus to trigger element on close (`onCloseAutoFocus`)
- ☐ Table has `aria-label`

Multi-Delete

- ☐ `Promise.allSettled` used (not `Promise.all`)
- ☐ Failed count reported to user via `toast.error`

Code Splitting

- ☐ Feature module uses `React.lazy(() => import(...))`
- ☐ Module is wrapped in `<Suspense fallback={...}>` in the router

10.3 Glossary

Term	Definition
DTO	Data Transfer Object. The exact shape of data from the server (PascalCase, raw types). Never contains UI concerns.
Row	Flattened, UI-ready version of a DTO. camelCase, formatted dates, display-friendly values. Co-located with its mapper.
Edit Context	Pre-populated data passed to the modal when editing an existing item. Built by <code>buildEditContext(dto)</code> .
Query Key	An array that uniquely identifies a React Query cache entry. Same key = shared cache.

Term	Definition
staleTime	How long React Query considers data "fresh" before re-fetching. <code>0</code> = immediately stale.
Discriminated Union	A TypeScript union type where one field (the "discriminant") distinguishes between variants. Enables type narrowing in <code>switch</code> statements.
Pure Function	A function with no side effects. Same input always produces same output. Trivially testable.
Orchestrator	A component that wires together other components but contains no business logic. The page component is the orchestrator.
Code Splitting	Splitting the JS bundle into chunks loaded on demand via <code>React.lazy</code> . Faster initial page load.
Controlled State	State managed externally and fed back in. TanStack Table's <code>state: { sorting } + onSortingChange: setSorting</code> pattern.
z.infer	Zod utility that extracts a TypeScript type from a schema. Types are always derived, never written manually.
flexRender	TanStack Table utility that renders a column's <code>header</code> or <code>cell</code> . Handles both function and plain value definitions.
satisfies	TypeScript operator that checks a type without widening it. Safer than <code>as</code> — catches errors while preserving the inferred type.
Promise.allSettled	Runs all Promises and waits for all to complete regardless of success or failure. Returns results array with <code>status: "fulfilled" or "rejected"</code> .
aria-live	HTML attribute that tells screen readers to announce content changes. <code>"polite"</code> waits for idle. <code>"assertive"</code> interrupts immediately.
useMemo	React hook that caches a computed value. Only recomputes when dependencies change. Prevents unnecessary re-renders.
useCallback	React hook that caches a function reference. Only creates a new function when dependencies change. Prevents child re-renders.
useReducer	React hook for managing coupled state. <code>(state, action) → newState</code> . Guarantees atomic updates.
FormProvider	React Hook Form context provider. Makes the form instance available to all child components via <code>useFormContext</code> .
useFormContext	React Hook Form hook. Reads the form instance from <code>FormProvider</code> context. No prop drilling needed.
Controller	React Hook Form component. Connects custom (non-HTML) inputs to the form via <code>value/onChange</code> interface.
Stale Closure	A bug where a function closes over a value from a previous render. Caused by missing dependencies in <code>useCallback/useMemo</code> .

Term	Definition
Co-location	Keeping related code in the same file. <code>ItemRow</code> and <code>mapItemDtoToRow</code> are co-located because they always change together.
Sentinel Value	A special value that signals "not set". <code>0</code> for <code>completedBy</code> means "use the logged-in user". <code>undefined</code> for <code>serverParams</code> means "user hasn't searched yet".
Nullish Coalescing (??)	Returns the right side only if the left is <code>null</code> or <code>undefined</code> . Unlike <code> </code> , it does NOT treat <code>0</code> or <code>""</code> as falsy.
Optional Chaining (?.)	Stops evaluation and returns <code>undefined</code> if the left side is <code>null</code> or <code>undefined</code> . <code>a?.b?.c</code> is safe even if <code>a</code> or <code>b</code> is null.
Immutable State Update	Returning a new state object instead of mutating the existing one. Required by React — mutation doesn't trigger re-renders.
Lazy-Load Boundary	The module file (<code>Crud.module.tsx</code>). Everything it imports is included in the lazy chunk. Keeps the main bundle small.
import type	Imports only the TypeScript type, not runtime code. The type is erased at compile time. Use for type-only imports to reduce bundle size.
as const	Makes array/object values readonly literal types. Enables TypeScript to catch typos in query key references at compile time.
getRowId	TanStack Table option that provides a stable row ID. Without it, TanStack uses row index — which changes when rows are filtered/sorted, breaking row selection.
initialState	TanStack Table option that sets starting state without making it controlled. The user can change it and the table remembers locally.
data-state	Shadcn's attribute for applying conditional styles. <code>data-state="selected"</code> on <code>TableRow</code> applies the selection highlight.
aria-invalid	HTML attribute that tells screen readers a form field has an error. Screen readers announce "invalid" when the field is focused.
aria-describedby	HTML attribute that links a form field to its description/error message by ID. Screen readers read the linked element when the field is focused.

10.4 Quick Decision Guide

"Should I use `useState` or `useReducer`?"

- Values that change independently → `useState`
- Values that change together (e.g., `modalOpen` + `editCtx`) → `useReducer`

"Should I use `useMemo` or `useCallback`?"

- Caching a computed VALUE → `useMemo`
- Caching a FUNCTION → `useCallback`

"Should I put this in the hook or the page?"

- Business logic, state, handlers, derived data → hook
- Layout, conditional rendering, wiring children together → page

"Should I put this in `dtos.ts` or `types.ts`?"

- Matches the backend API shape → `dtos.ts`
- UI-only concept (edit context, form values) → `types.ts`

"Should I use `z.infer` or write the type manually?"

- Always `z.infer`. Never write types manually when a Zod schema exists.

"Should I use `Promise.all` or `Promise.allSettled`?"

- All-or-nothing (one failure should cancel all) → `Promise.all` (rare)
- Best-effort (partial success is acceptable) → `Promise.allSettled` (multi-delete)

"Should I add `zodParse` to this service function?"

- GET (reading from server) → yes, always
- POST/PUT/DELETE (writing to server) → no, we control the payload

"Should I add `staleTime` to this query?"

- Reference data that rarely changes (categories, years, settings) → yes, 5–10 minutes
- User data that changes frequently (student list) → no (default 0)

You have completed the *CRUD v4 In-Depth Guide*. Return to [index.md](#) for the overview.